Low-Latency Joint Sensing Architecture for Bionic Hands Using Piezo Sensor

Category: Robotics and Intelligent Machines

Table of Contents

Section	Page
1. Abstract	1
2. Introduction	2
3. Literature Review	3
4. Methodology	3
5. Results	5
6. Discussion	5
7. Conclusion	6
8. Acknowledgement & references	7

Abstract

Bionic hands are essential for restoring grip and dexterity, but their response time is limited by two separate sources of delay. The motor itself takes approximately 200 milliseconds to respond, and this is a fixed mechanical limit. On the other side, ADC sampling and digital filtering add another 80–150 milliseconds before the motor even starts to move. While the mechanical delay cannot be reduced easily, the software - processing delay can be eliminated. This project aims to reduce that delay using a purely hardware-based signal path.

We implemented a piezoelectric disc sensor to detect touch and impact force. Instead of routing this signal through the microcontroller's software for processing, we passed it through an LM393 analog comparator, which directly triggers a microcontroller hardware interrupt, eliminating the software polling loop entirely. The electronic decision-path latency was calculated using manufacturer datasheet values (LM393 propagation delay: 1.3 microseconds; microcontroller interrupt response: 0.3–1 microseconds). This estimate was experimentally validated using a hardware testbed. The measured values (~9 microseconds for both modes) were somewhat higher than the theoretical estimate due to real-world execution overhead, but remained consistent with the overall conclusion.
Our calculations show this design's electronic decision-path latency at approximately 2 microseconds, which is significantly faster, by several orders of magnitude, than published response times for existing prosthetic tactile-sensing systems. These systems typically show response times between 20 milliseconds (in ML-based grasp-detection systems) and 100–200 milliseconds (in advanced e-skin/pressure-sensor designs). This result is specific: it does not make the whole hand faster overall, and it does not replace AI-based grip or force-control processing. Instead, it functions as a small, low-cost, low-weight hardware addition built for fast-reaction critical situations, such as sudden slip or impact, operating as a rapid, reflex-like pathway alongside existing intelligent control systems.

1. Introduction

Whether used in a prosthetic hand or a robotic hand, artificial hands need a fast response to touch, impact, or slip. A delay in detecting these events can reduce grip stability and affect performance in real situations.
The motor takes approximately 200ms for a physical move; this is a fixed mechanical limit and cannot be reduced. In addition to this, the software processing (sensor data collection) typically takes another 80–150ms, because the software runs a continuous loop to check for new data, which adds a delay every cycle.
This software-processing delay becomes a bottleneck in emergency-type situations, such as a sudden slip or unexpected impact, where even a small reduction in detection time can improve grip stability and safety — which is exactly what this project investigates: whether the software-processing delay can be eliminated using a purely hardware-based approach, without changing the motor's mechanical response time.
To address this, we used a hardware-based sensing method. In this method, a piezoelectric sensor, an LM393 comparator, and a microcontroller hardware interrupt are used to completely bypass the software polling loop for the detection step.

2. Literature Review

Bionic and prosthetic hands are currently focusing on better sensing, such as tactile sensors. 1X Neo uses a tendon-driven design with multiple tactile sensors. On the other side, some advanced systems like Bebionic use machine learning, reducing response time, and they have reported tactile response time around 20 milliseconds using LSTM-based grasp prediction. At the same time, many advanced e-skin and pressure-sensor designs still show response time between 100 to 200 milliseconds. Among the closer prior work, SmartHand (IEEE, 2022) is particularly relevant. In this system, they used a resistive sensor combined with CNN-based machine learning to achieve high-throughput data at approximately 100 frames per second. The main goal of SmartHand is to know about recognition or force estimation. As a result, the system is software-heavy and depends on continuous processing. In contrast, our project has a different objective: instead of focusing on classification or continuous data richness, it uses a piezo sensor connected to an LM393 comparator that directly triggers a microcontroller hardware interrupt and completely bypasses the software polling loop. It is low-cost and specifically aimed at reducing detection latency rather than increasing sensing intelligence.Overall, most existing research has focused on improving the intelligence of tactile sensing through software and machine-learning techniques. Less attention has been given to minimizing the electronic decision-path delay for quick, reflex-like responses. This project addresses this narrow but fills an important gap by using a purely hardware-based method for fast event detection.

3. Methodology

The circuit was assembled on a breadboard using a piezoelectric sensor, an LM393 comparator, a potentiometer, and an Arduino Uno. The output of the piezoelectric sensor was connected to the input of the LM393 comparator, and a potentiometer was used to set the reference threshold voltage of the comparator. The output of the LM393 was then connected to digital pin 2 (INT0) of the Arduino, which supports hardware interrupts. Proper 5V power and ground connections were provided to all components. The potentiometer was adjusted to set a suitable threshold level on the LM393. This threshold was important for noise rejection, so that small unwanted vibrations would not trigger the system — only when a genuine touch or impact produced a signal above the set threshold would the comparator output go high.

In initial testing, manual tapping was used to test the system, but the force and timing of each tap were different, made accurate comparison difficult. To improve consistency, a controlled digital trigger was used. A separate Arduino pin (Pin 4) generated a consistent pulse instead of manual tapping. This ensured that both Mode A and Mode B were tested under the same conditions.

Two detection methods were implemented and compared:

- **Mode A (Software Polling):** In this mode, the program continuously checks the sensor signal using a loop, even when no event is happening. This continuous checking is what causes the extra delay.
- **Mode B (Hardware Interrupt):** In this mode, the attachInterrupt() function was used. As soon as the comparator output went high, the microcontroller was interrupted directly, without needing to continuously check the loop, which significantly reduced the delay.

Response time was measured using Arduino's micros() function, and we ran 15 trials for both modes. The average value was calculated, and the data was recorded through the Serial Monitor.

In addition to experimental testing, a theoretical calculation of the electronic decision-path latency was performed using manufacturer datasheet values. The following table summarizes the theoretical latency for each stage:

Table 1: Datasheet-based latency breakdown for the electronic decision-path.

Stage	Value	Source
FPC/wire propagation	Negligible (~ns)	Formula-based
LM393 comparator delay	1.3 µs	Datasheet (TI/ON Semi)
MCU interrupt response	0.3–1 µs	Datasheet (ATmega328)
Total	**~1.6–2.3 µs**	**Sum**

4. Results

The latency of Mode A (Software Polling) and Mode B (Hardware Interrupt) was measured using the hardware testbed. Fifteen trials were conducted for each mode. The average latency was 9 microseconds for Mode A and 9 microseconds for Mode B. Most trial values ranged between 8 and 12 microseconds.

According to the datasheet-based calculation presented in the Methodology section, the theoretical electronic decision-path latency is approximately 2 microseconds. The measured values were higher than this theoretical estimate. This difference is expected, due to real-world execution overhead in the microcontroller.

Even though the experimental values are higher than the theoretical calculation, they remain significantly lower than the response times reported for existing prosthetic tactile-sensing systems. Published systems typically show latencies between 20 milliseconds (in ML-based grasp-detection approaches) and 100–200 milliseconds (in advanced e-skin and pressure-sensor designs).

In this controlled test, both modes produced similar results, likely because the microcontroller was handling a simple, single task with very little overhead. However, this does not fully represent how the system would behave under a busier workload — for example, when the microcontroller is also handling motor control alongside sensor monitoring. In such conditions, the difference between Mode A and Mode B is expected to increase. Mode A would likely slow down since it must wait its turn to check the sensor, while Mode B would still respond immediately because interrupts are handled with priority.

5. Discussion

The experimental results showed that both Mode A (software polling) and Mode B (hardware interrupt) produced an average latency of approximately 9 microseconds. The values were similar because the Arduino Uno is relatively fast, and when the microcontroller is under a zero-load condition, even simple polling does not take much time. The measured values for both modes were higher than the theoretical estimate of about 2 microseconds. This difference is expected, because real hardware involves additional execution overhead, such as function calls and code processing. Mode A was used as an example of a basic software-polling approach, which is still common in many low-cost and educational systems. The aim was to compare continuous software checking with a direct hardware-interrupt method.

Even though Mode A and Mode B performed similarly in this test, Mode B still has an important architectural advantage: a hardware interrupt can respond as soon as a signal arrives and does not depend on how busy the microcontroller is. In contrast, software polling can become slower when the microcontroller is managing multiple tasks at the

same time, such as motor control or sensor filtering. This difference did not appear clearly in the present experiment, because the microcontroller was focused only on one task. Future tests, in which the microcontroller is kept busy with other tasks, would help show this advantage more clearly.

Although the published response times of 20–200 milliseconds generally include additional processing, such as object recognition or force estimation, the latency measured in this work (about 9 microseconds) is much lower. This shows that a simple hardware-based method can detect critical events, like sudden slip or impact, much faster than typical software-based systems.

There are also some limitations. The present work is a single-sensor proof of concept and has not yet been scaled to a full multi-joint hand. Testing was done when the microcontroller was free; in a real bionic hand, the microcontroller is busy. Small variations in measurement are also possible due to timer and interrupt timing.

6. Conclusion

This project explored whether a hardware-based approach, using a piezoelectric sensor, an LM393 comparator and a microcontroller interrupt, could reduce the response-time latency for detecting sudden slip or impact situations in bionic hands. The measured latency was approximately 9 microseconds, which is significantly lower than the 20–200 millisecond range reported for existing prosthetic tactile-sensing systems. This method does not reduce the overall mechanical response time of the hand or replace AI-based grip control; it is intended as a small hardware addition for fast reaction in critical situations, such as sudden slip or impact. In a real bionic hand, this interrupt-based path can work alongside the main control system and quickly trigger a reflex-like response when needed. In future, this hardware-based detection method can be extended to multiple finger joints in a prosthetic or robotic hand, where piezoelectric sensors can be placed near the joints to detect sudden contact or slip events and testing it under more realistic situations, where the microcontroller is busy with other tasks.

7. Acknowledgement & References

My teammate built and tested the electronic testbed and collected the measurements. My teacher reviewed and signed the approval and safety forms. AI tools were used as follows: Claude, Grok and Gemini for explaining electronics concepts and checking calculations; Claude and Gemini for Arduino test code; Claude and Grok for drafting and language editing of report sections; Claude for literature search and the Figure 1 diagram. The project idea, circuit design, testing and conclusions are ours.

References:

1. 1X Technologies. NEO's Hands | An API to the Physical World. www.1x.tech/discover/neos-hands
2. LM393 Comparator Datasheet (Texas Instruments/ON Semiconductor)
3. ATmega328 Microcontroller Datasheet (external interrupt specifications)
4. Leveraging LSTM, tactile sensors, and haptic feedback to augment prosthetic control via grasp type prediction. Scientific Reports, 2025. www.nature.com/articles/s41598-025-92651-z
5. At First Contact: Stiffness Estimation Using Vibrational Information for Prosthetic Grasp Modulation. arxiv.org/pdf/2411.18507
6. Tactile-Driven Gentle Grasping for Human-Robot Collaborative Tasks. arxiv.org/pdf/2303.09346
7. Wang, X., Geiger, F., Niculescu, V., Magno, M., & Benini, L. "Leveraging Tactile Sensors for Low Latency Embedded Smart Hands for Prosthetic and Robotic Applications. IEEE Transactions on Instrumentation and Measurement, vol. 71, pp. 1–14, 2022, Art no. 4004314, doi: 10.1109/TIM.2022.3165828. arxiv.org/pdf/2203.15069